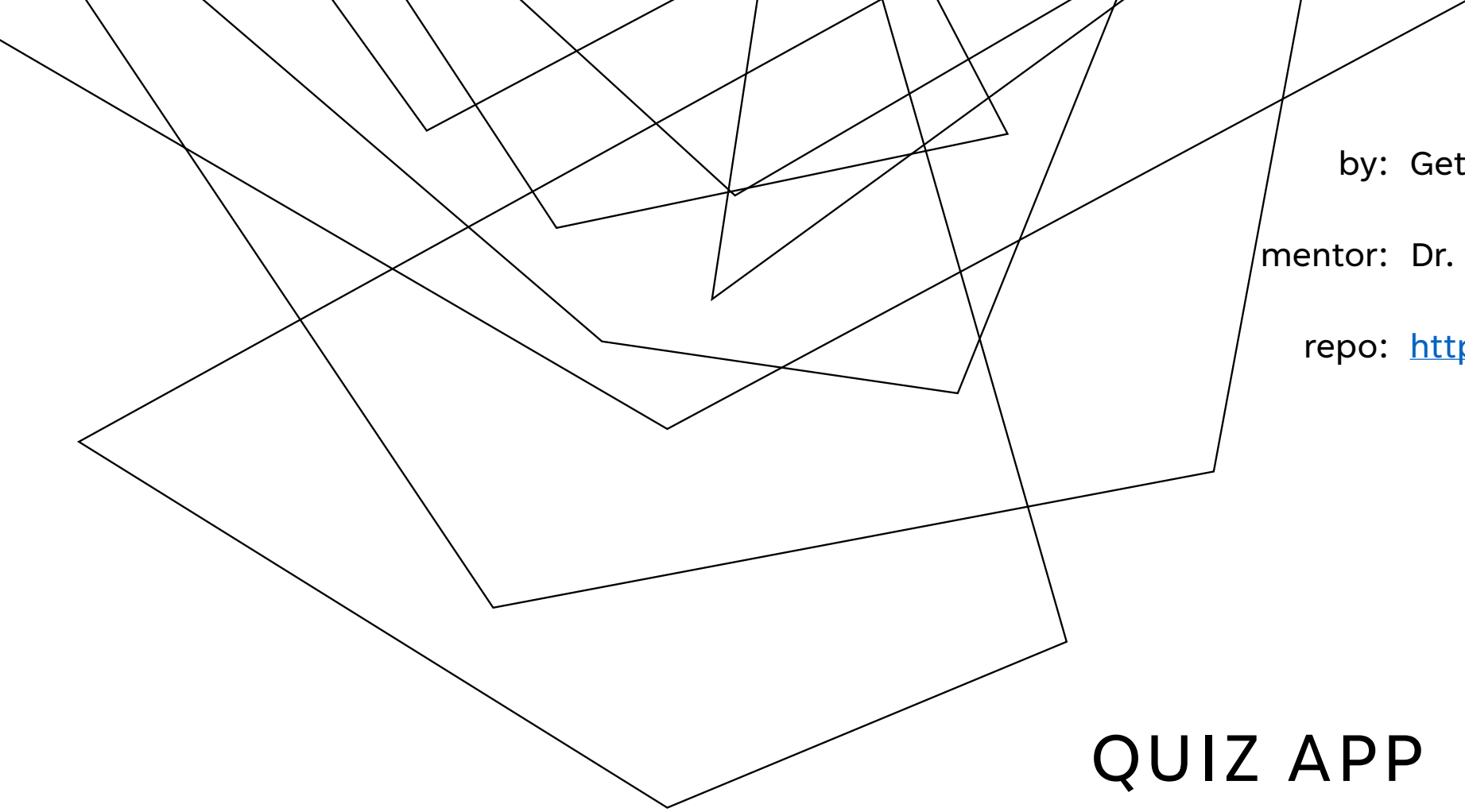


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern on the left side of the slide.

by: Getoar Krasniqi

mentor: Dr. Sc. Ermira Daka

repo: <https://github.com/Geti23/quiz-app>

QUIZ APP

built with **Test Driven Development**

AGENDA

- Overview
- Feature 1
- Feature 1 - Refactor
- Feature 2
- Feature 2 - Refactor
- Feature 3
- Feature 3 - Refactor
- Some Testing Statistics

Core Quiz Engine with support for multiple choice questions

Advanced Features including timed quizzes, difficulty levels, and category-based organization

Persistent Database with complete CRUD operations

REST API with 10+ endpoints using FastAPI

100% Test Coverage with 61 test cases across 11 test files

Production-Ready Code with type hints, validation, and documentation

OVERVIEW

*showcase app usage via Swagger

Question class with validation and answer checking

Quiz class with question management and scoring

Result class with percentage calculation and pass/fail logic

12 unit tests with 96% coverage

RED PHASE

```
coverage: platform win32, python 3.13.7-final-0
===== short test summary info =====
FAILED tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow - assert None is not None
FAILED tests/test_question.py::TestQuestion::test_create_multiple_choice_question - AttributeError: 'Question' object has no attribute 'text'
FAILED tests/test_question.py::TestQuestion::test_check_correct_answer - AssertionError: assert None == True
FAILED tests/test_question.py::TestQuestion::test_question_requires_text - Failed: DID NOT RAISE <class 'ValueError'>
FAILED tests/test_quiz.py::TestQuiz::test_create_empty_quiz - AttributeError: 'Quiz' object has no attribute 'title'
FAILED tests/test_quiz.py::TestQuiz::test_add_question_to_quiz - AttributeError: 'Quiz' object has no attribute 'questions'
FAILED tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions - AttributeError: 'Quiz' object has no attribute 'questions'
FAILED tests/test_quiz.py::TestQuiz::test_get_question_by_index - assert None == <src.question.Question object at 0x00000188A15BAE00>
FAILED tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score - assert None is not None
FAILED tests/test_result.py::TestQuizResult::test_perfect_score - assert None == True
FAILED tests/test_result.py::TestQuizResult::test_passing_grade - assert None == True
FAILED tests/test_result.py::TestQuizResult::test_result_summary - assert None is not None
===== 12 failed in 0.63s =====
```

FEATURE 1

GREEN PHASE

```
PS C:\Users\DELL\Documents\GitHub\quiz-app> pytest --cov
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.1, pluggy-1.6.0 -- C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DELL\Documents\GitHub\quiz-app
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.11.0, cov-7.0.0
collected 12 items

tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED [ 8%]
tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED [ 16%]
tests/test_question.py::TestQuestion::test_check_correct_answer PASSED [ 25%]
tests/test_question.py::TestQuestion::test_question_requires_text PASSED [ 33%]
tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED [ 41%]
tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED [ 50%]
tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED [ 58%]
tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED [ 66%]
tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED [ 75%]
tests/test_result.py::TestQuizResult::test_perfect_score PASSED [ 83%]
tests/test_result.py::TestQuizResult::test_passing_grade PASSED [ 91%]
tests/test_result.py::TestQuizResult::test_result_summary PASSED [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.7-final-0

Name          Stmts  Miss  Cover   Missing
-----
src\__init__.py    4     0  100%
src\question.py   15     2   87%   19, 26
src\quiz.py       22     0  100%
src\result.py     11     0  100%
-----
TOTAL             52     2   96%
Coverage HTML written to dir htmlcov
===== 12 passed in 0.39s =====
```

Refactoring of feature 1 consisted mainly adding helper methods to clean out the code and make it more readable. Also added more comments codebase wide and brought the coverage to 100%.

After refactoring the test results were not affected, which is the point of this phase in TDD.

*showcase full refactoring on VS Code

FEATURE 1 - REFACTOR

BEFORE

```
4 class Question:
5     """Represents a single quiz question with multiple choice options"""
6
7     def __init__(
8         self,
9         text: str,
10        options: List[str],
11        correct_answer: str,
12        difficulty: str = "medium",
13        category: Optional[str] = None,
14    ) -> None:
15-     if not text or text.strip() == "":
16-         raise ValueError("Question text cannot be empty")
17-
18     self.text = text
19     self.options = options
20     self.correct_answer = correct_answer
21     self.difficulty = difficulty
22     self.category = category
23
```

AFTER

```
4 class Question:
5     """Represents a single quiz question with multiple choice options"""
6
7     def __init__(
8         self,
9         text: str,
10        options: List[str],
11        correct_answer: str,
12        difficulty: str = "medium",
13        category: Optional[str] = None,
14    ) -> None:
15+     self._validate_text(text)
16
17     self.text = text
18     self.options = options
19     self.correct_answer = correct_answer
20     self.difficulty = difficulty
21     self.category = category
22
23+ @staticmethod
24+ def _validate_text(text: str) -> None:
25+     """Validate that question text is not empty"""
26+     if not text or text.strip() == "":
27+         raise ValueError("Question text cannot be empty")
```

FEATURE 2

Timed quiz functionality (time limits, elapsed time tracking)

Question difficulty levels (easy, medium, hard) with filtering

Category-based organization with score breakdown by category

Answer review system (track incorrect answers, detailed feedback)

RED PHASE

```
tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED
tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED
tests/test_question.py::TestQuestion::test_check_correct_answer PASSED
tests/test_question.py::TestQuestion::test_question_requires_text PASSED
tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED
tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED
tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED
tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED
tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED
tests/test_quiz_categories.py::TestQuizCategories::test_question_has_category FAILED
tests/test_quiz_categories.py::TestQuizCategories::test_quiz_can_get_questions_by_category FAILED
tests/test_quiz_categories.py::TestQuizCategories::test_quiz_get_score_by_category FAILED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_has_difficulty_level FAILED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_default_difficulty_is_medium FAILED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_filter_questions_by_difficulty FAILED
tests/test_quiz_review.py::TestQuizReview::test_get_incorrect_answers FAILED
tests/test_quiz_review.py::TestQuizReview::test_get_answer_details FAILED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_has_time_limit FAILED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_starts_timer_on_first_answer FAILED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_tracks_elapsed_time FAILED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_can_check_if_time_expired FAILED
tests/test_result.py::TestQuizResult::test_perfect_score PASSED
tests/test_result.py::TestQuizResult::test_passing_grade PASSED
tests/test_result.py::TestQuizResult::test_result_summary PASSED
```

GREEN PHASE

```
tests/test_integration.py::TestQuizIntegration::test_complete_quiz_flow PASSED
tests/test_question.py::TestQuestion::test_create_multiple_choice_question PASSED
tests/test_question.py::TestQuestion::test_check_correct_answer PASSED
tests/test_question.py::TestQuestion::test_question_requires_text PASSED
tests/test_quiz.py::TestQuiz::test_create_empty_quiz PASSED
tests/test_quiz.py::TestQuiz::test_add_question_to_quiz PASSED
tests/test_quiz.py::TestQuiz::test_cannot_add_duplicate_questions PASSED
tests/test_quiz.py::TestQuiz::test_get_question_by_index PASSED
tests/test_quiz.py::TestQuiz::test_submit_answer_and_track_score PASSED
tests/test_quiz_categories.py::TestQuizCategories::test_question_has_category PASSED
tests/test_quiz_categories.py::TestQuizCategories::test_quiz_can_get_questions_by_category PASSED
tests/test_quiz_categories.py::TestQuizCategories::test_quiz_get_score_by_category PASSED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_has_difficulty_level PASSED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_question_default_difficulty_is_medium PASSED
tests/test_quiz_difficulty.py::TestQuestionDifficulty::test_filter_questions_by_difficulty PASSED
tests/test_quiz_review.py::TestQuizReview::test_get_incorrect_answers PASSED
tests/test_quiz_review.py::TestQuizReview::test_get_answer_details PASSED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_has_time_limit PASSED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_starts_timer_on_first_answer PASSED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_tracks_elapsed_time PASSED
tests/test_quiz_timer.py::TestQuizTimer::test_quiz_can_check_if_time_expired PASSED
tests/test_result.py::TestQuizResult::test_perfect_score PASSED
tests/test_result.py::TestQuizResult::test_passing_grade PASSED
tests/test_result.py::TestQuizResult::test_result_summary PASSED
```


FEATURE 2 - REFACTOR

Refactoring of feature 2 code without affecting the test results (still all passing with 100% cov)

BEFORE

```
def get_score_by_category(self, category: Optional[str]) -> Dict[str, float]:
    """Get the score for questions in a specific category"""
    category_questions = []
    score = 0

    for index, question in enumerate(self.questions):
        if question.category == category:
            category_questions.append(index)
            if index in self.answers and question.check_answer(self.answers[index]):
                score += 1

    total = len(category_questions)
    percentage = (score / total * 100) if total > 0 else 0.0

    return {"score": score, "total": total, "percentage": percentage}
```

AFTER

```
113 def get_score_by_category(self, category: Optional[str]) -> Dict[str, float]:
114     """Get the score for questions in a specific category"""
115+     category_indices = self._get_category_question_indices(category)
116+
117+     score = self._calculate_category_score(category_indices)
118
119+     total = len(category_indices)
120+     percentage = (score / total * 100) if total > 0 else 0.0
121+
122+     return {"score": score, "total": total, "percentage": percentage}
123+
124+ def _get_category_question_indices(self, category: Optional[str]) -> List[int]:
125+     """Get indices of all questions in a category"""
126+     return [
127+         index for index, question in enumerate(self.questions) if question.category == category
128+     ]
129+
130+ def _calculate_category_score(self, indices: List[int]) -> int:
131+     """Calculate score for a list of question indices"""
132+     score = 0
133+     for index in indices:
134+         if index in self.answers and self._is_answer_correct(index, self.answers[index]):
135+             score += 1
136+     return score
137+
138+ def __repr__(self) -> str:
139+     """String representation for debugging"""
140+     return f"Quiz(title='{self.title}', questions={len(self.questions)})"
```

RED PHASE

```
tests/test_quiz_database.py::TestQuizDatabase::test_create_quiz_in_database FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_read_quiz_from_database FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_update_quiz_in_database FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_delete_quiz_from_database FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_list_all_quizzes FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_get_nonexistent_quiz FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_update_nonexistent_quiz FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_delete_nonexistent_quiz FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_database_generates_unique_ids FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_clear_database FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_is_stored_in_quiz_object FAILED
tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_persists_after_update FAILED
```

short test summary info

```
FAILED tests/test_api.py::TestHealthEndpoints::test_root_endpoint_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestHealthEndpoints::test_health_endpoint_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_returns_created - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_minimal_data_returns_created - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_invalid_data_returns_error - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestReadQuizEndpoints::test_get_quiz_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestReadQuizEndpoints::test_get_nonexistent_quiz_returns_not_found - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestReadQuizEndpoints::test_list_quizzes_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestReadQuizEndpoints::test_list_quizzes_when_empty_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestUpdateQuizEndpoint::test_update_quiz_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestUpdateQuizEndpoint::test_update_nonexistent_quiz_returns_not_found - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestUpdateQuizEndpoint::test_update_quiz_with_invalid_data_returns_error - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestDeleteQuizEndpoint::test_delete_quiz_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestDeleteQuizEndpoint::test_delete_nonexistent_quiz_returns_not_found - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestDeleteQuizEndpoint::test_clear_all_quizzes_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_to_nonexistent_quiz_returns_not_found - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_with_invalid_index_returns_bad_request - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestResultsEndpoint::test_get_results_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestResultsEndpoint::test_get_results_for_nonexistent_quiz_returns_not_found - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestAPIDocumentationEndpoints::test_swagger_docs_returns_ok - assert 404 == 200
FAILED tests/test_api.py::TestAPIDocumentationEndpoints::test_redoc_docs_returns_ok - assert 404 == 200
FAILED tests/test_api.py::TestAPIDocumentationEndpoints::test_openapi_json_returns_ok - assert 404 == 200
FAILED tests/test_api.py::TestCRUDWorkflow::test_complete_crud_workflow_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
FAILED tests/test_api.py::TestQuizCompletionWorkflow::test_complete_quiz_taking_workflow_returns_ok - fastapi.exceptions.ResponseValidationError: 1 validation errors:
===== 25 failed in 21.35s =====
```

GREEN PHASE

```
tests/test_quiz_database.py::TestQuizDatabase::test_create_quiz_in_database PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_read_quiz_from_database PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_update_quiz_in_database PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_delete_quiz_from_database PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_list_all_quizzes PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_get_nonexistent_quiz PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_update_nonexistent_quiz PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_delete_nonexistent_quiz PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_database_generates_unique_ids PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_clear_database PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_is_stored_in_quiz_object PASSED
tests/test_quiz_database.py::TestQuizDatabase::test_quiz_id_persists_after_update PASSED
tests/test_api.py::TestHealthEndpoints::test_root_endpoint_returns_ok PASSED
tests/test_api.py::TestHealthEndpoints::test_health_endpoint_returns_ok PASSED
tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_returns_created PASSED
tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_minimal_data_returns_created PASSED
tests/test_api.py::TestCreateQuizEndpoint::test_create_quiz_with_invalid_data_returns_error PASSED
tests/test_api.py::TestReadQuizEndpoints::test_get_quiz_returns_ok PASSED
tests/test_api.py::TestReadQuizEndpoints::test_get_nonexistent_quiz_returns_not_found PASSED
tests/test_api.py::TestReadQuizEndpoints::test_list_quizzes_returns_ok PASSED
tests/test_api.py::TestReadQuizEndpoints::test_list_quizzes_when_empty_returns_ok PASSED
tests/test_api.py::TestUpdateQuizEndpoint::test_update_quiz_returns_ok PASSED
tests/test_api.py::TestUpdateQuizEndpoint::test_update_nonexistent_quiz_returns_not_found PASSED
tests/test_api.py::TestUpdateQuizEndpoint::test_update_quiz_with_invalid_data_returns_error PASSED
tests/test_api.py::TestDeleteQuizEndpoint::test_delete_quiz_returns_ok PASSED
tests/test_api.py::TestDeleteQuizEndpoint::test_delete_nonexistent_quiz_returns_not_found PASSED
tests/test_api.py::TestDeleteQuizEndpoint::test_clear_all_quizzes_returns_ok PASSED
tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_returns_ok PASSED
tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_to_nonexistent_quiz_returns_not_found PASSED
tests/test_api.py::TestAnswerSubmissionEndpoint::test_submit_answer_with_invalid_index_returns_bad_request PASSED
tests/test_api.py::TestResultsEndpoint::test_get_results_returns_ok PASSED
tests/test_api.py::TestResultsEndpoint::test_get_results_for_nonexistent_quiz_returns_not_found PASSED
tests/test_api.py::TestAPIDocumentationEndpoints::test_swagger_docs_returns_ok PASSED
tests/test_api.py::TestAPIDocumentationEndpoints::test_redoc_docs_returns_ok PASSED
tests/test_api.py::TestAPIDocumentationEndpoints::test_openapi_json_returns_ok PASSED
tests/test_api.py::TestCRUDWorkflow::test_complete_crud_workflow_returns_ok PASSED
tests/test_api.py::TestQuizCompletionWorkflow::test_complete_quiz_taking_workflow_returns_ok PASSED
```

FEATURE 3

QuizDatabase with full CRUD operations

12 tests with 100% code coverage

10+ REST endpoints with full CRUD operations

Pydantic models for request/response validation

Automatic API documentation (Swagger UI, ReDoc)

Health check and info endpoints

25 API endpoint tests

100% code coverage across all modules

FEATURE 3 - REFACTOR

Updated all Pydantic models to use **ConfigDict** instead of the deprecated **Config** class.

Dependency Injection - Added `get_db()` function and passed database through `Depends()` instead of using the global `db`. This makes testing easier and follows FastAPI best practices.

DRY Principle - Extracted repeated quiz creation logic into `_create_quiz_from_model()` function, eliminating duplication between create and update endpoints.

Error Handling Helper - Created `_get_quiz_or_404()` to centralize the quiz lookup and 404 handling, reducing repetitive code across endpoints.

Response Models - Added `AnswerResponseModel` and `QuizResultsModel` Pydantic classes for type safety and automatic validation of response data.

Consistent Naming - Prefixed internal helper functions with underscore (`_quiz_to_dict`, etc.) to make the public API clearer.

After all this change and refactoring the test results and coverage were left unchanged

BEFORE

```
@app.post("/quizzes", response_model=QuizResponseModel, status_code=201)
async def create_quiz(quiz_data: QuizCreateModel) -> QuizResponseModel:

    """
    CREATE - Add a new quiz to the database.

    Returns the created quiz with its generated ID.
    """

    # Create Quiz object
    quiz = Quiz(title=quiz_data.title, time_limit_seconds=quiz_data.time_limit_seconds)

    # Add questions
    for q_data in quiz_data.questions:
        question = Question(
            text=q_data.text,
            options=q_data.options,
            correct_answer=q_data.correct_answer,
            difficulty=q_data.difficulty,
            category=q_data.category,
        )
        quiz.add_question(question)

    # Store in database
    quiz_id = db.add_quiz(quiz)
```

AFTER

```
@app.post("/quizzes", response_model=QuizResponseModel, status_code=201)
async def create_quiz(
    quiz_data: QuizCreateModel, database: QuizDatabase = Depends(get_db)
) -> QuizResponseModel:

    """
    CREATE - Add a new quiz to the database.

    Returns the created quiz with its generated ID.
    """

    quiz = _create_quiz_from_model(quiz_data)
    quiz_id = database.add_quiz(quiz)

# Helper functions
def _create_quiz_from_model(quiz_data: QuizCreateModel) -> Quiz:
    """Factory function to create a Quiz object from a request model"""
    quiz = Quiz(title=quiz_data.title, time_limit_seconds=quiz_data.time_limit_seconds)
    for q_data in quiz_data.questions:
        question = Question(
            text=q_data.text,
            options=q_data.options,
            correct_answer=q_data.correct_answer,
            difficulty=q_data.difficulty,
            category=q_data.category,
        )
        quiz.add_question(question)
    return quiz
```

*showcase full refactoring on VS Code

SOME TESTING STATISTICS

Categories	UNIT TESTS	INTEGRATION TESTS	END-TO-END TESTS	TOTAL
	42 (68.85%)	17 (27,87%)	2 (3.28%)	61 (100%)
Coverage	MODULE	STATEMENTS	EXECUTED	COVERAGE
	question.py	19	19	100%
	quiz.py	70	70	100%
	result.py	14	14	100%
	database.py	35	35	100%
	api.py	98	98	100%
	TOTAL	241	241	100%

A series of white, thin, overlapping geometric lines on a black background, forming a complex, abstract pattern on the left side of the slide.

THANK YOU

Questions?